Contents

1 Terminal Management System

Flask API for the Mellon Group DevOps Bootcamp final assignment. The local stack runs with Docker Compose and includes:

- `tms-api`: Flask application built from `app/Dockerfile`
- `mysql`: official MySQL image with persistent data and automatic schema/seed import
- `redis`: official Redis image used as the cache layer

1.1 Project Layout

```
.
├── app/
│   ├── Dockerfile
│   ├── main.py
│   └── requirements.txt
├── db/
│   └── init/
│       ├── 01_schema.sql
│       └── 02_seed.sql
├── docker-compose.yml
├── .env.example
└── README.md
```

1.2 Environment

Create a local `.env` file from the template:

```
cp .env.example .env
```

Update the password values before running the stack. The real `.env` file is ignored by git.

1.3 Run

```
docker compose up --build
```

The API will be available at:

```
http://localhost:5000
```

Useful endpoints:

- `GET /health`: checks both MySQL and Redis connectivity and returns `503` if either component is degraded
- `GET /schema/terminals`: runs the schema exploration query from the assignment
- `GET /terminals`: lists all terminals
- `GET /terminals?enabled=true`: lists enabled terminals
- `GET /terminals?enabled=false`: lists decommissioned/disabled terminals
- `GET /templates`: lists templates
- `GET /templates/<id>`: returns one template
- `POST /terminals/from-template`: creates a terminal from a template and merchant MID
- `GET /terminals/<tid>`: returns details for one terminal
- `GET /terminals/flagged`: lists terminals with a non-zero scenario number
- `POST /terminals/<tid>/flag`: sets `scenario_number` and updates `updated_on`
- `POST /terminals/<tid>/unflag`: sets `scenario_number` to `0` and updates `updated_on`
- `POST /terminals/<tid>/decommission`: disables a terminal and queues it for deletion after 3 days
- `GET /terminals/decommissioned`: lists terminals in the decommission queue
- `GET /statistics/by-hardware`: terminal count by hardware model
- `GET /statistics/by-state`: active/inactive terminal counts
- `GET /statistics/by-hardware-family`: terminal count by hardware family
- `GET /statistics/idle-distribution`: terminal count by idle-days bucket

Application logs are written to stdout with timestamp, level, and message.

1.4 Redis Caching

The API uses a cache-aside pattern with Redis. `GET /terminals` is cached for 30 seconds, and `GET /statistics/*` endpoints are cached for 60 seconds. Cache operations are best-effort: if Redis is unavailable, the API logs the Redis error and continues by reading from MySQL.

Every write endpoint clears the whole application cache before returning:

- `POST /terminals/<tid>/flag`
- `POST /terminals/<tid>/unflag`
- `POST /terminals/<tid>/decommission`
- `POST /terminals/from-template`

To verify cache behavior, watch the API logs:

```
docker compose logs -f tms-api
```

In another terminal:

```
curl http://localhost:5000/terminals
curl http://localhost:5000/terminals
curl -X POST http://localhost:5000/terminals/T0101001/flag \
  -H "Content-Type: application/json" \
  -d '{"scenario_number":"5"}'
curl http://localhost:5000/terminals
```

The expected log flow is `Cache MISS`, then `Cache HIT`, then cache clear after the write, then `Cache MISS` again.

1.5 Database Initialization

The SQL files in `db/init/` are mounted to `/docker-entrypoint-initdb.d` inside the MySQL container. The official MySQL image executes them automatically only when the database volume is created for the first time.

To recreate the database from scratch:

```
docker compose down -v
docker compose up --build
```

1.6 Schema Note

The `mid` field belongs to the `merchants` table. The `terminals` table stores `merchant_id`, which references `merchants.id`; therefore a terminal's MID is retrieved through a join.

On API startup, the app applies the Feature A schema additions idempotently:

- Adds `terminals.updated_on` only if the column does not already exist.
- Creates `decommission_queue` only if the table does not already exist.

1.7 Feature A Examples

List terminals:

```
curl http://localhost:5000/terminals
curl "http://localhost:5000/terminals?enabled=true"
```

Terminal details:

```
curl http://localhost:5000/terminals/T0101001
```

Flag and unflag:

```
curl -X POST http://localhost:5000/terminals/T0101001/flag \
  -H "Content-Type: application/json" \
  -d '{"scenario_number":"5"}'

curl -X POST http://localhost:5000/terminals/T0101001/unflag
```

Decommission:

```
curl -X POST http://localhost:5000/terminals/T0101001/decommission
curl http://localhost:5000/terminals/decommissioned
```

1.8 Feature B Examples

List templates:

```
curl http://localhost:5000/templates
```

Template details:

```
curl http://localhost:5000/templates/1
```

Create terminal from template:

```
curl -X POST http://localhost:5000/terminals/from-template \
  -H "Content-Type: application/json" \
  -d '{"template_id":1,"mid":"MID000101"}'
```

The create operation runs in a single database transaction. It validates the template, validates the merchant MID, locks the merchant's existing terminals, calculates the next TID suffix, inserts the new terminal, and returns `201 Created`.

1.9 Feature D Examples

Statistics endpoints use Pandas for aggregation and Redis cache with a 60-second TTL:

```
curl http://localhost:5000/statistics/by-hardware
curl http://localhost:5000/statistics/by-state
curl http://localhost:5000/statistics/by-hardware-family
curl http://localhost:5000/statistics/idle-distribution
```

1.10 Data Schema Check

Use this query to inspect the `terminals` table:

```sql
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'terminals'
ORDER BY ORDINAL_POSITION;
```

After the API starts, the result includes the assignment seed columns plus the `updated_on` column added by the application migration.